

HƯỚNG DẪN GHI AI_AUDITLOG

Topic: Orders Management System

Course: Programming Fundamentals (PRF19x, PCP291)

Dành cho sinh viên lập trình cơ bản với C/C++ ghi AI Audit Log theo RBL Insight Framework và Domain Thinking Components

Nguồn tham chiếu

- *AI_AuditLog_Student_Guideline.pdf; AI_AuditLog_Template.xlsx.*
- *Các DTC Components dùng trong hướng dẫn: Decomposition, Pattern Recognition, Abstraction, Algorithms.*
- *Assignment-OrdersManagementSystem.docx: Product gồm product name, price, quantity; hệ thống cần add, update, delete, list available, search by name, sort by price ascending, print all, save to file.*

Mục tiêu tài liệu

- Giúp sinh viên biết prompt nào cần ghi vào AI_AuditLog và prompt nào chỉ là hỗ trợ phụ.
- Gắn từng prompt với đúng Domain Thinking Component (DTC): Decomposition, Pattern Recognition, Abstraction, Algorithms.
- Cung cấp bộ prompt mẫu cho topic Orders Management System (OMS) với C/C++.
- Hướng dẫn viết Human Delta & Reflection đủ 4 ý: Critical Thinking, Contextualization, Creative Synthesis, Decision Ownership.
- Gợi ý các hallucination cases và evidence phù hợp để sinh viên chuẩn bị oral viva.

1. Mục tiêu của topic

Topic “Orders Management System” không chỉ là viết menu **thêm/sửa/xóa sản phẩm**. Sinh viên cần chứng minh mình hiểu bản chất bài toán **quản lý danh sách sản phẩm, xử lý dữ liệu hợp lệ, thực hiện truy vấn và lưu dữ liệu ra file**.

- Dữ liệu cần **quản lý** là danh sách **Product** gồm: **product name, product price và product quantity**.

- Các chức năng chính: **add new product, update product, delete product, list available products quantity > 0, search by name, sort by price ascending, print all, save to file.**
- Trong C/C++, sinh viên cần chọn cách biểu diễn **Product**, cách **lưu** danh sách sản phẩm và cách **tách hàm/module** cho rõ ràng.
- Cần quyết định giữa **struct array + count, vector<Product>, class Store** hoặc **linked list** tùy ngôn ngữ và yêu cầu môn học.
- Cần xử lý edge cases: **store rỗng, trùng tên sản phẩm, sản phẩm không tìm thấy, price/quantity không hợp lệ, tên có khoảng trắng, delete làm lệch chỉ số, sort sau khi update, lỗi ghi file.**

Thông điệp chính

AI Audit Log phải thể hiện sinh viên tự ra quyết định thiết kế, phản biện AI và kiểm chứng thuật toán. Không ghi toàn bộ lịch sử chat, không ghi các prompt chỉ sửa syntax hoặc format code.

2. Cách phân biệt Core Prompt và Auxiliary Prompt

2.1. Core Prompt

Một prompt được xem là **CORE** nếu **không có prompt đó thì project có thể thay đổi về cấu trúc dữ liệu, cách phân rã module, thiết kế Product/Store, thuật toán add/update/delete/search/sort, xử lý edge cases hoặc phương pháp kiểm chứng độ đúng** của chương trình.

Ví dụ CORE

- “For an Orders Management System for beginner programming, should I store products in a fixed array, dynamic list/vector, dictionary/map, or linked list? Compare them based on add, update, delete, search by name, sort by price, and save to file.”

2.2. Auxiliary Prompt

Một prompt là **AUXILIARY** nếu nó **chỉ giúp sửa lỗi nhỏ, format code, đặt tên biến hoặc hỏi cú pháp cơ bản**. Những prompt này có thể dùng để học nhưng không nên ghi vào Audit Log.

Ví dụ AUXILIARY

- “How to write a for loop?”
- “Fix my missing semicolon.”
- “Rename variable p to product.”
- “Translate comments to English.”

Bảng phân biệt nhanh Core/Auxiliary Prompt

Loại Prompt	CORE - Nên ghi	AUXILIARY - Không ghi
Decision-Making	So sánh cấu trúc lưu trữ Product cho add/update/delete/search/sort/save .	Hỏi cách khai báo biến, vòng lặp hoặc import thư viện.
Problem-Solving	Thiết kế thuật toán delete product không làm mất dữ liệu và cập nhật count/index đúng.	Nhờ sửa lỗi thiếu dấu ‘;’, thiếu dấu ‘:’, lỗi block lệnh hoặc lỗi thụt lề.
Verification	Kiểm tra sort/search/save có xử lý store rõ ràng, trùng tên, giá bằng nhau, lỗi ghi file không.	Nhờ thêm comment, đổi tên biến, làm đẹp menu.

3. Số lượng entry nên ghi

Với topic này, nên ghi khoảng 10-12 core entries. Con số này dựa trên trung bình số quyết định thiết kế quan trọng của một project nhập môn: 2 quyết định decomposition, 2 pattern recognition, 2 abstraction, 4-6 thuật toán chính và ít nhất 2 hallucination cases. Không nên ghi quá ít vì thiếu minh chứng tư duy; cũng không nên ghi quá nhiều prompt nhỏ vì dễ bị đánh giá là copy chat.

- Decomposition: ít nhất 1-2 core prompts.
- Pattern Recognition: ít nhất 1-2 core prompts.
- Abstraction: ít nhất 1-2 core prompts.
- Algorithms: ít nhất 2-4 core prompts.
- Hallucination Detection: ít nhất 2 cases.

4. Bảng Core Prompts đề xuất cho Sheet “2. Detailed Audit Log”

Các prompt dưới đây là mẫu định hướng. Sinh viên nên chỉnh lại theo project/code thật của mình và ghi evidence cụ thể.

Entry	Prompt Type	DTC	Problem/Context	Prompt to AI	Vì sao là Core + Evidence
001	DECISION	Decomposition	Cần chia OMS thành module nhỏ trước khi code.	Break down an Orders Management System in C/C++ into core modules: Product data model, store storage, add/update/delete, search, sort, display, validation, menu, and file saving . Which modules are essential for a beginner-level project?	Core: Ảnh hưởng cách tổ chức toàn bộ project theo hàm/module. Evidence: sơ đồ module, checklist chức năng.
002	PROBLEM-SOLVING	Decomposition	Cần xác định chức năng bắt buộc và chức năng mở rộng.	For this OMS assignment, which features are core requirements and which are optional extensions: add, update, delete, list available quantity > 0, search by name, sort by price ascending, print all, save to file, load from file, statistics?	Core: Giúp giới hạn scope đúng yêu cầu đề. Evidence: bảng core/optional features.
003	DECISION	Pattern Recognition	Nhận diện mẫu CRUD + query + persistence.	What common programming pattern exists in this Orders Management System? Explain how add, update, delete, search, sort, print, and save fit into CRUD, query/filter, ordering, and persistence patterns .	Core: Giúp nhận ra cấu trúc bài toán, không viết rời rạc từng hàm. Evidence: ghi chú pattern CRUD/query/sort/save.
004	VERIFICATION	Pattern Recognition	Cần kiểm tra edge cases lặp lại ở nhiều chức năng.	List repeated edge cases for add/update/delete/search/sort/save in a product store: empty store, duplicate name, not found, invalid price, invalid quantity, product name with spaces,	Core: Giúp phát hiện pattern biên và tái sử dụng validation. Evidence: bảng edge cases theo chức năng.

Entry	Prompt Type	DTC	Problem/Context	Prompt to AI	Vì sao là Core + Evidence
				file open failure. Which functions must handle each case?	
005	DECISION	Abstraction	Cần thiết kế Product đúng kiểu dữ liệu.	How should I design a Product struct/class in C/C++ with name, price, and quantity? What validation rules should belong to Product and what should belong to Store functions?	Core: Ảnh hưởng mô hình dữ liệu. Evidence: struct Product hoặc class Product, constraints.
006	DECISION	Abstraction	Cần tách Store management khỏi menu.	How should I abstract store management so main/menu only calls addProduct(), updateProduct(), deleteProduct(), searchByName(), sortByPrice(), printAll(), saveToFile(), instead of directly modifying the array/vector everywhere?	Core: Ảnh hưởng encapsulation và giảm lỗi sửa count/index. Evidence: function list hoặc Store class diagram.
007	DECISION	Algorithms	Cần chọn cấu trúc lưu trữ Product.	For a beginner OMS in C/C++, compare fixed array + count, dynamic array/vector, linked list, and map by name. Which is best for add, update, delete, search by name, sort by price ascending, and save to file?	Core: Quyết định cấu trúc dữ liệu quan trọng nhất. Evidence: bảng so sánh data structures + quyết định cuối.
008	PROBLEM-SOLVING	Algorithms	Cần viết addProduct đúng validation.	Design the addProduct algorithm in C/C++. Handle full array/capacity, duplicate product name, name with spaces, price <= 0, quantity < 0, and successful insertion. Give pseudocode before code.	Core: Ảnh hưởng correctness chức năng add. Evidence: pseudocode addProduct, test data hợp lệ/không hợp lệ.

Entry	Prompt Type	DTC	Problem/Context	Prompt to AI	Vì sao là Core + Evidence
009	PROBLEM-SOLVING	Algorithms	Cần update/delete không làm mất dữ liệu.	Design updateProduct() and deleteProduct() for an array/vector of Product. How should I find the product by name, update fields safely, and delete by shifting elements or vector erase while keeping count/list consistent?	Core: Case dễ lỗi index/count. Evidence: trace delete first/middle/last/not found.
010	DECISION	Algorithms	Cần sắp xếp Product theo price tăng dần.	For sorting products by price ascending in C/C++, should I implement bubble/selection/insertion sort manually or use std::sort? Compare based on course requirement, readability, and time complexity. How should equal prices be handled?	Core: Ảnh hưởng thuật toán sort và độ phức tạp. Evidence: pseudocode sort, output trước/sau sort.
011	VERIFICATION	Algorithms	Cần kiểm chứng search by name.	Verify a searchByName() design for C/C++. Should it use exact match, partial match, or case-insensitive comparison? How to avoid false positives and handle names with spaces?	Core: Ảnh hưởng kết quả truy vấn. Evidence: test search exact/partial/not found/tên có khoảng trắng.
012	VERIFICATION	Algorithms	Cần lưu file đúng định dạng và kiểm chứng output.	Design and verify saveToFile() for products in C/C++. Should I save as CSV or formatted text? How should I handle names with spaces, delimiters, file open failure, and verify the file content after saving?	Core: Persistence là yêu cầu bắt buộc. Evidence: file output mẫu, screenshot, test file open failure.

5. Mẫu Human Delta & Reflection cho các entry quan trọng

5.1. Entry 007 - Data Structure Decision

Field	Nội dung mẫu
Prompt Type	DECISION
Stage/Component	Algorithms
Problem/Context	Cần chọn cấu trúc lưu trữ Product cho OMS triển khai bằng C/C++.
Prompt to AI	For a beginner OMS in C/C++, compare fixed array + count, dynamic array/vector, linked list, and map by name. Which is best for add, update, delete, search by name, sort by price ascending, and save to file?
AI Response Summary	AI so sánh fixed array, vector, linked list và map. AI gợi ý vector trong C++ vì dễ thêm/xóa/sort; với C cơ bản, fixed array + count phù hợp nếu đề không yêu cầu cấp phát động.
Human Delta & Reflection	Critical Thinking: AI đúng khi khuyên dùng vector cho C++, nhưng nếu lớp đang học C cơ bản thì vector không dùng được. Linked list không tối ưu cho sort và làm bài phức tạp hơn. Contextualization: Scope assignment chỉ yêu cầu quản lý một danh sách sản phẩm nhỏ, có search/sort/save, không yêu cầu dữ liệu rất lớn. Creative Synthesis: Tôi tách quyết định theo ngôn ngữ: C dùng struct Product products[MAX] + count; C++ dùng vector<Product>. Decision Ownership: Tôi chọn cấu trúc đơn giản nhất phù hợp môn học để dễ trace delete/sort và dễ chuẩn bị oral viva.
Evidence	Bảng so sánh fixed array, vector, linked list, map; pseudocode add/delete/sort; test với 0, 1 và nhiều sản phẩm.

5.2. Entry 008 - addProduct Algorithm

Field	Nội dung mẫu
Prompt Type	PROBLEM-SOLVING
Stage/Component	Algorithms
Problem/Context	Cần viết addProduct không nhận dữ liệu sai và không làm tràn mảng.
Prompt to AI	Design the addProduct algorithm in C/C++. Handle full array/capacity, duplicate product name, name with spaces, price ≤ 0 , quantity < 0 , and successful insertion. Give pseudocode before code.
AI Response Summary	AI đề xuất kiểm tra capacity, nhập name/price/quantity, kiểm tra trùng tên, kiểm tra price và quantity, sau đó thêm Product vào danh sách.
Human Delta & Reflection	Critical Thinking: AI xử lý đúng luồng chính nhưng ban đầu chưa nhấn mạnh name có khoảng trắng và lỗi nhập số.

Field	Nội dung mẫu
	Contextualization: Trong C/C++, nhập chuỗi bằng cin >> name hoặc scanf("%s") sẽ cắt tên sản phẩm tại khoảng trắng. Creative Synthesis: Tôi bổ sung getline/fgets cho name và quy tắc price > 0, quantity >= 0. Decision Ownership: Tôi quyết định validate trước khi tăng count hoặc push_back để tránh thêm sản phẩm lỗi vào Store.
Evidence	Pseudocode addProduct, test case: name trùng, price âm, quantity âm, tên “Milk Powder”, store đầy.

5.3. Entry 010 - Sort by Price Ascending

Field	Nội dung mẫu
Prompt Type	DECISION
Stage/Component	Algorithms
Problem/Context	Cần sắp xếp danh sách sản phẩm theo giá tăng dần.
Prompt to AI	For sorting products by price ascending in C/C++, should I implement bubble/selection/insertion sort manually or use std::sort? Compare based on course requirement, readability, and time complexity. How should equal prices be handled?
AI Response Summary	AI gợi ý C++ có thể dùng std::sort với comparator theo price, còn C có thể dùng thuật toán thủ công như selection sort hoặc bubble sort nếu đang học nhập môn.
Human Delta & Reflection	Critical Thinking: AI nói std::sort tốt hơn về hiệu năng nhưng tôi cần kiểm tra yêu cầu môn học; nếu yêu cầu tự cài thuật toán thì không nên dùng sẵn. Contextualization: Bài tập lập trình cơ bản cần chứng minh hiểu vòng lặp, đổi chỗ Product và so sánh price. Creative Synthesis: Tôi chọn selection sort cho C hoặc khi cần minh họa thuật toán; chọn std::sort cho C++ nếu giáo viên cho phép. Decision Ownership: Tôi giữ rule sort tăng dần theo double price và không đổi thứ tự không cần thiết khi price bằng nhau nếu dùng stable approach.
Evidence	Output trước/sau sort, test hai sản phẩm cùng giá.

6. Gợi ý Hallucination Detection cho topic này

Các case này nên ghi ở Sheet “**3. Hallucination Detection**”. Mỗi case cần có AI’s Claim, Reality Check, How Detected và Corrective Action.

Entry	Hallucination Type	AI’s Claim	Reality Check	How Detected	Corrective Action
007	Context Misunderstanding	Use Python list/dictionary for the C/C++ assignment.	Sai ngôn ngữ ; nếu bài yêu cầu C/C++ thì không dùng cấu trúc Python.	Đối chiếu code AI với môi trường C/C++.	Chuyển sang struct array, vector<Product> hoặc class Store phù hợp C/C++.
008	Oversimplification	Use scanf("%s", name) or cin >> name for product name.	Sai nếu tên sản phẩm có khoảng trắng , ví dụ “Milk Powder”.	Test nhập tên có khoảng trắng và thấy bị cắt chuỗi.	Dùng fgets trong C hoặc getline trong C++ sau khi xử lý newline.
009	Logic Error	Delete product by setting quantity = 0.	Đó là đánh dấu hết hàng, không phải xóa khỏi Store ; print all vẫn còn sản phẩm.	Chạy print all sau delete.	Với array thì shift phần tử và giảm count; với vector thì erase.
010	Library/Function Fabrication	Use products.sort() in C++ vector.	std::vector không có member sort() ; chỉ list có sort, vector dùng std::sort trong <algorithm>	Compile code và nhận lỗi “no member named sort”.	Dùng std::sort(products.begin(), products.end(), comparator).
012	Oversimplification	Save by writing binary struct directly to file.	Dễ lỗi portability, tên chuỗi/spacing, khó đọc và khó kiểm tra bằng mắt .	Mở file output thấy dữ liệu không thân thiện hoặc lỗi với string.	Dùng CSV/text có delimiter rõ ràng; kiểm tra file sau khi lưu.

7. Auxiliary Prompts không nên ghi vào Audit Log

Prompt	Vì sao không nên ghi?	Ghi chú
How to declare int count in C?	Chỉ hỏi cú pháp biến cơ bản.	Không ghi log.
Fix missing semicolon in line 25.	Sửa lỗi syntax nhỏ, không ảnh hưởng thiết kế.	Không ghi log.
What is #include <iostream>?	Kiến thức cú pháp/thư viện cơ bản.	Không ghi log.
Make my menu prettier with cout.	UI phụ, không ảnh hưởng thuật toán chính.	Không ghi trừ khi quyết định flow menu lớn.
Rename Product p to product.	Đổi tên biến, không đổi bản chất.	Không ghi log.
Add comments to every line.	Hỗ trợ trình bày, không phải core decision.	Không ghi log.
Why does Dev-C++ show red underline?	Debug môi trường, không liên quan DTC.	Không ghi trừ khi lỗi ảnh hưởng thuật toán.
Convert cout to printf.	Chuyển cú pháp I/O, không thay đổi thiết kế.	Không ghi log.
Generate 5 sample products.	Dữ liệu test đơn giản, không phải decision.	Chỉ dùng làm evidence.
Format the saved file with tabs.	Format output nhỏ.	Không ghi nếu không ảnh hưởng khả năng đọc/parse file.
How to declare int count in C?	Chỉ hỏi cú pháp biến cơ bản.	Không ghi log.

8. Gợi ý điền Sheet “1. Metadata & Summary”

DTC Component	Số prompt nên có	Gợi ý Entry
Decomposition	2	001, 002
Pattern Recognition	2	003, 004
Abstraction	2	005, 006
Algorithms	6	007, 008, 009, 010, 011, 012

AI Usage Summary nên ghi

- Total Prompts Used: tổng số prompt thật sự đã dùng với AI, ví dụ 40-80.

- Core Prompts Logged: chỉ đếm prompt quan trọng, ví dụ 10-12.
- Hallucination Detected: ít nhất 2 cases.
- Selection Ratio: số core prompts / tổng prompts; không nên quá cao vì log không phải copy toàn bộ chat.

9. Checklist tự kiểm tra trước khi nộp

#	Tiêu chí	Pass?
1	Prompt này có ảnh hưởng đến quyết định quan trọng trong project không?	☐
2	Nếu bỏ prompt này, project có thay đổi về design/architecture/algorithm không?	☐
3	Tôi có phản biện hoặc kiểm chứng AI không?	☐
4	Tôi có evidence cụ thể không: pseudocode, code snippet, test case, trace table, UML?	☐
5	Human Delta có đủ 4 phần: Critical Thinking, Contextualization, Creative Synthesis, Decision Ownership không?	☐

Quy tắc loại bỏ entry

Nếu một entry chỉ là “AI viết code cho tôi và tôi copy vào”, entry đó không đạt. Nếu entry không pass ít nhất 4/5 tiêu chí, nên loại bỏ hoặc viết lại.

10. Evidence phù hợp cho topic này

- **BẮT BUỘC (phải có trong mỗi entry):**
 - [BẮT BUỘC] Pseudocode hoặc code snippet minh họa quyết định chính.
 - [BẮT BUỘC] Test case cụ thể (ít nhất 1 valid + 1 edge case) có input và expected output.
- **KHUYẾN KHÍCH (tăng điểm evidence):**
 - Bảng so sánh cấu trúc dữ liệu: fixed array + count, vector<Product>, linked list, map.
 - Sơ đồ module/hàm: Product, addProduct, updateProduct, deleteProduct, searchByName, sortByPrice, printAll, listAvailable, saveToFile, menu.
 - Pseudocode add/update/delete/search/sort/save.
 - Trace delete product ở vị trí đầu, giữa, cuối và not found.
 - Test cases: empty store, one product, duplicate name, invalid price, invalid quantity, name with spaces, equal prices, quantity = 0.
 - Output terminal trước/sau sort và trước/sau delete.

- File output CSV/text sau saveToFile và ảnh/screenshot kiểm tra nội dung file.

11. Quy tắc quyết định cuối cùng cho sinh viên

11.1 Với C cơ bản

Nên dùng struct Product products[MAX] và biến count. Cách này phù hợp nhập môn, dễ trace, dễ kiểm tra capacity, dễ minh họa xóa bằng shifting và sort thủ công.

11.2 Với C++ cơ bản

Nên dùng struct/class Product kết hợp vector<Product>. Vector hỗ trợ push_back, erase, duyệt, sort với std::sort nếu được phép; code gọn hơn nhưng vẫn cần sinh viên giải thích được thuật toán và comparator.

11.3 Khi nào không nên dùng linked list

Không nên chọn linked list chỉ vì nghe “linh hoạt”. Với OMS cần sort theo price, search by name và save toàn bộ danh sách, array/vector thường đơn giản và phù hợp hơn cho project nhập môn lập trình cơ bản.

12. Mẫu Reflection cuối log

Trong project Orders Management System bằng C/C++, AI hỗ trợ tôi phân rã bài toán thành các hàm, so sánh cấu trúc dữ liệu, thiết kế thuật toán add/update/delete/search/sort và kiểm tra file saving. Tuy nhiên, tôi không copy trực tiếp output của AI. Tôi dùng AI như công cụ phản biện và kiểm chứng. Quyết định cuối cùng của tôi là dùng struct Product với array + count cho C hoặc vector<Product> cho C++ vì phù hợp scope, dễ triển khai và dễ kiểm thử. Tôi cũng phát hiện một số lỗi AI như dùng sai thư viện/ngôn ngữ, bỏ qua tên sản phẩm có khoảng trắng, xóa sản phẩm bằng cách đặt quantity = 0, hoặc dùng products.sort() cho vector C++. Các lỗi này được sửa bằng validation, trace thủ công và test cases.

Điểm cần nhấn mạnh khi hướng dẫn sinh viên

Đừng ghi prompt hỏi cú pháp. Hãy ghi prompt làm thay đổi cách phân rã, nhận dạng pattern, trừu tượng hóa thiết kế hoặc thuật toán. Mỗi entry cần có evidence cụ thể và phản ánh tư duy phản biện.

13. Hướng dẫn viết AI Response Summary đúng cách

Cột "AI Response Summary" trong Audit Log **KHÔNG** được paste nguyên văn output của AI. Sinh viên cần paraphrase lại bằng ngôn ngữ của mình — tức là viết lại hoàn toàn ý chính, không giữ nguyên câu chữ. Mục đích là chứng minh sinh viên đã đọc hiểu, không phải copy.

13.1 Nguyên tắc paraphrase

- **Chỉ ghi ý chính: 2–4 câu tóm gọn điều AI đề xuất/gợi ý.**
- **Không copy nguyên đoạn code dài của AI vào ô Summary — code đó thuộc về phần Evidence.**
- **Dùng ngôi thứ ba khi mô tả AI: "AI đề xuất...", "AI gợi ý...", "AI so sánh..."**

13.2 Ví dụ so sánh: Viết đúng vs Viết sai

- **SAI - Paste nguyên văn AI:**

"To implement a product store in C++, you can use std::vector<Product> which provides dynamic resizing, O(1) amortized push_back, and integrates seamlessly with std::sort via a custom comparator..."

- **ĐÚNG — Paraphrase bằng ngôn ngữ của mình:**

"AI khuyên dùng vector<Product> cho C++ vì hỗ trợ thêm/xóa linh hoạt và dễ sort bằng std::sort. AI cũng lưu ý rằng với C cơ bản, fixed array + count phù hợp hơn."

14. Ví dụ entry CHƯA ĐẠT — để đối chiếu và tránh lặp lại

Dưới đây là ví dụ một entry điển hình **KHÔNG ĐẠT** để sinh viên nhận biết và tránh. So sánh với mẫu đạt ở Mục 5.

14.1 Ví dụ entry CHƯA ĐẠT

- **Prompt to AI:** "Write me the addProduct function in C++."
- **AI Response Summary:** [Paste nguyên đoạn code 50 dòng từ AI vào đây]
- **Human Delta & Reflection:** "AI viết đúng, tôi dùng luôn."
- **Evidence:** [Để trống]

14.2 Phân tích lỗi của entry trên

- Prompt không có context/constraints: không nêu edge cases, không hỏi pseudocode, không yêu cầu giải thích.
- AI Response Summary paste nguyên code — không thể hiện sinh viên đã đọc hiểu.

- Human Delta chỉ 1 câu, không có *Critical Thinking, Contextualization, Creative Synthesis hay Decision Ownership*.
- Evidence để trống — không có gì chứng minh sinh viên đã kiểm chứng output.
- Kết quả: entry này không pass tiêu chí nào trong checklist Mục 9. Nên viết lại toàn bộ theo mẫu Entry 008 ở Mục 5.

15. Rubric chấm điểm AI Audit Log

Rubric dưới đây giúp giảng viên chấm nhất quán từng entry trong Audit Log. Mỗi entry tối đa 4 điểm thành phần.

15.1 Rubric theo từng entry

- **Prompt (1 điểm):** Đủ (1đ) = có context, constraints, yêu cầu rõ. Thiếu (0đ) = prompt ngắn chung chung, không có DTC rõ ràng.
- **AI Response Summary (1 điểm):** Đủ (1đ) = paraphrase rõ ý AI, 2–4 câu. Thiếu (0đ) = paste nguyên văn hoặc để trống.
- **Human Delta & Reflection (1 điểm):** Đủ (1đ) = có đủ 4 phần Critical Thinking / Contextualization / Creative Synthesis / Decision Ownership, mỗi phần ít nhất 1–2 câu thực chất. Thiếu (0.5đ) = chỉ có 2–3 phần hoặc phần nào đó chung chung. Không đạt (0đ) = chỉ 1 câu hoặc copy AI.
- **Evidence (1 điểm):** Đủ (1đ) = có ít nhất 1 evidence bắt buộc (pseudocode hoặc test case). Tốt (1đ bonus) = có thêm trace table, Big-O, screenshot. Thiếu (0đ) = để trống.

15.2 FAQ – các câu hỏi thường gặp

- **H: Sinh viên dùng ChatGPT thay vì Claude thì ghi Audit Log thế nào?**
➔ Đ: Audit Log không phụ thuộc vào công cụ AI cụ thể. Sinh viên ghi đúng **Prompt, AI Response Summary, Human Delta và Evidence** là đủ. Có thể thêm cột "AI Tool Used" nếu cần phân biệt.
- **H: Nếu AI trả lời sai hoàn toàn thì vẫn ghi log không?**
➔ Đ: Vẫn ghi — đây là **Hallucination case**. Sinh viên ghi vào **Sheet 3**, nêu rõ **AI's Claim, Reality Check và Corrective Action**. **Đây là một trong những entry có giá trị cao nhất.**
- **H: Sinh viên hỏi AI rất nhiều nhưng chỉ ghi 10 entry — có bị nghi ngờ gian lận không?**
➔ Đ: **Không**. Selection Ratio thấp (10 core / 60 total = 17%) là bình thường và đúng. **Điều quan trọng là 10 entry đó phải thật sự là core decisions, có evidence và Human Delta đủ 4 phần.**
- **H: Entry mà sinh viên tự nghĩ ra không dùng AI — có ghi không?**

- ➔ Đ: Audit Log chỉ ghi các prompt thực sự đã gửi AI. Quyết định tự đưa ra (không dùng AI) được thể hiện qua **Human Delta** — ghi rõ **"Tôi tự quyết định... không tham khảo AI vì..."**